

HR Insights

HR Insights is a Python-based HR analytics platform built around a medallion-style ETL pipeline. It ingests raw employee and timesheet files, loads them through bronze and silver data layers, computes gold-layer KPIs, **generates interactive visualizations** that tell a data story, and exposes curated data through a FastAPI application.

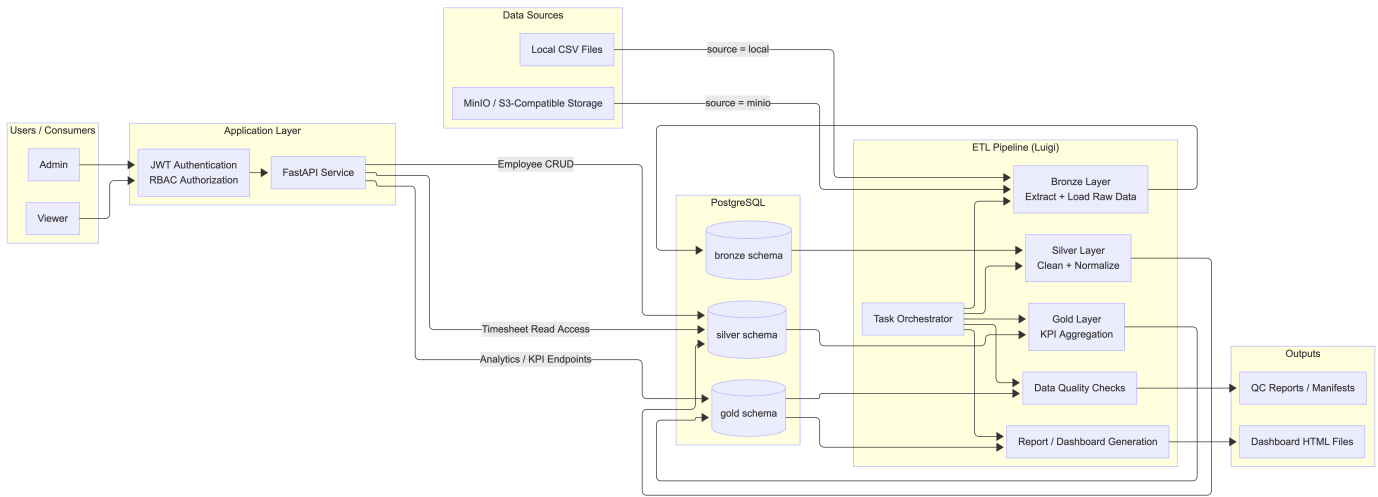
Table of Contents

- 1. [Overview](#)
- 2. [Engineering Decisions](#)
- 3. [Setup Instructions](#)
- 4. [Usage Guide](#)
- 5. [Visualizations](#)
- 6. [Schema Documentation](#)
- 7. [API Documentation](#)
- 8. [Project Structure](#)
- 9. [Environment Variables](#)
- 10. [Analytics SQL](#)

Overview

- **ETL**: Luigi-based tasks discover files from local storage or MinIO, load bronze raw tables, normalize silver entities, validate quality, and compute gold metrics.
- **API**: FastAPI provides CRUD access for employees and read-oriented endpoints for timesheets and analytics.
- **Storage**: PostgreSQL stores the bronze, silver, and gold schemas. MinIO is supported as an object-storage source for raw CSV ingestion.
- **Visualizations**: Plotly generates interactive HTML dashboards from gold analytics—clear, actionable insights that tell a data story, with filters, drill-downs, and time ranges.

Architecture



Engineering Decisions

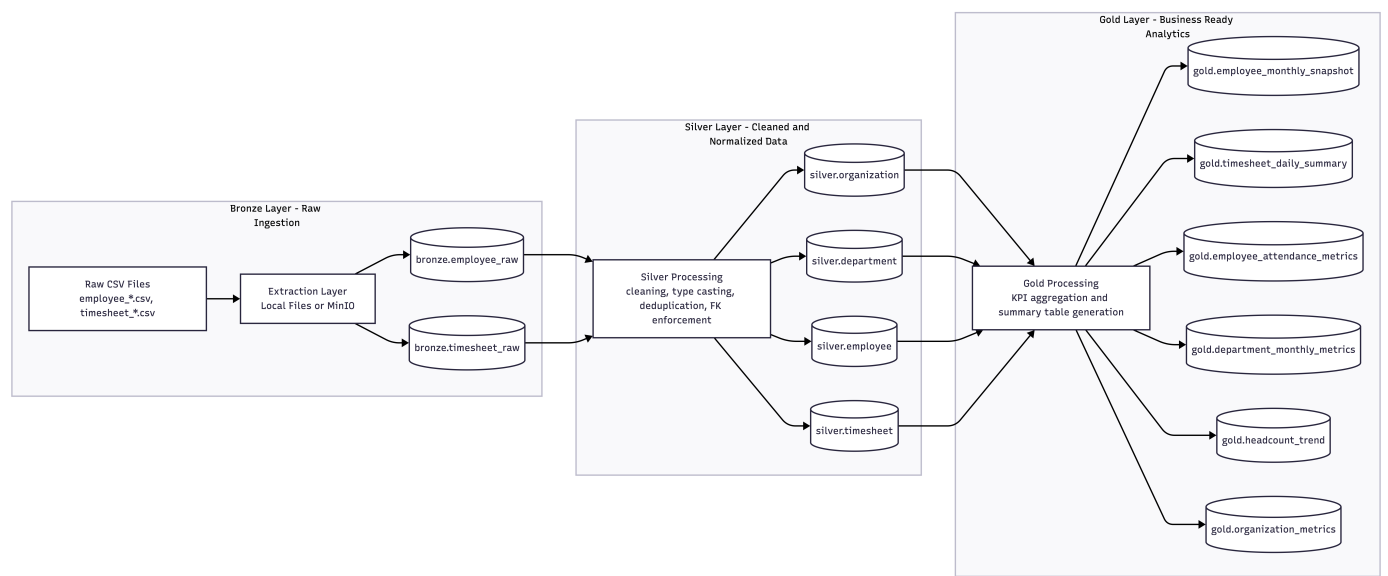
Technology Choices

Area	Choice	Why it was chosen	Trade-off
Language	Python 3.10+	One stack covers ETL, API development, scripting, and reporting through pandas, SQLAlchemy, Luigi, FastAPI, and Plotly.	Runtime performance is lower than compiled alternatives, but iteration speed is better for this workload.
Orchestration	Luigi	The pipeline is dependency-driven and fits Luigi tasks well. Luigi also provides simple local scheduling and filesystem-based completion markers.	It is lighter-weight than Airflow or Prefect and has less built-in observability.

Area	Choice	Why it was chosen	Trade-off
Database	PostgreSQL	The project needs relational integrity for employees, departments, organizations, and timesheets, plus support for multiple schemas.	More normalization work is required before data is analytics-ready.
Raw storage	MinIO	Docker deployments need an S3-compatible source for raw files without introducing external cloud dependencies.	It adds a service compared with a local-files-only workflow.
ORM / migrations	SQLAlchemy 2.x + Alembic	The same stack is reused by both the API and ETL, and Alembic handles schema evolution cleanly.	Models and migrations must be kept aligned manually.
API	FastAPI	Typed request/response schemas and automatic OpenAPI docs fit the service layer well.	It introduces a second runtime surface area beyond the ETL pipeline.
Reporting	Plotly HTML dashboards	Reports need to be easy to generate and share without a separate BI tool.	Static HTML is not a multi-user analytics platform.

Architecture Decisions

Medallion architecture



The project separates data into three schemas:

- **bronze**: raw landing tables with minimal transformation and `source_file` lineage.
- **silver**: cleaned, deduplicated, typed, and relational tables used by the API and downstream transformations.
- **gold**: precomputed analytical tables designed for fast reporting and stable KPI queries.

This improves traceability and rerun safety. The trade-off is that some entities exist in multiple layers, which increases storage and transformation complexity.

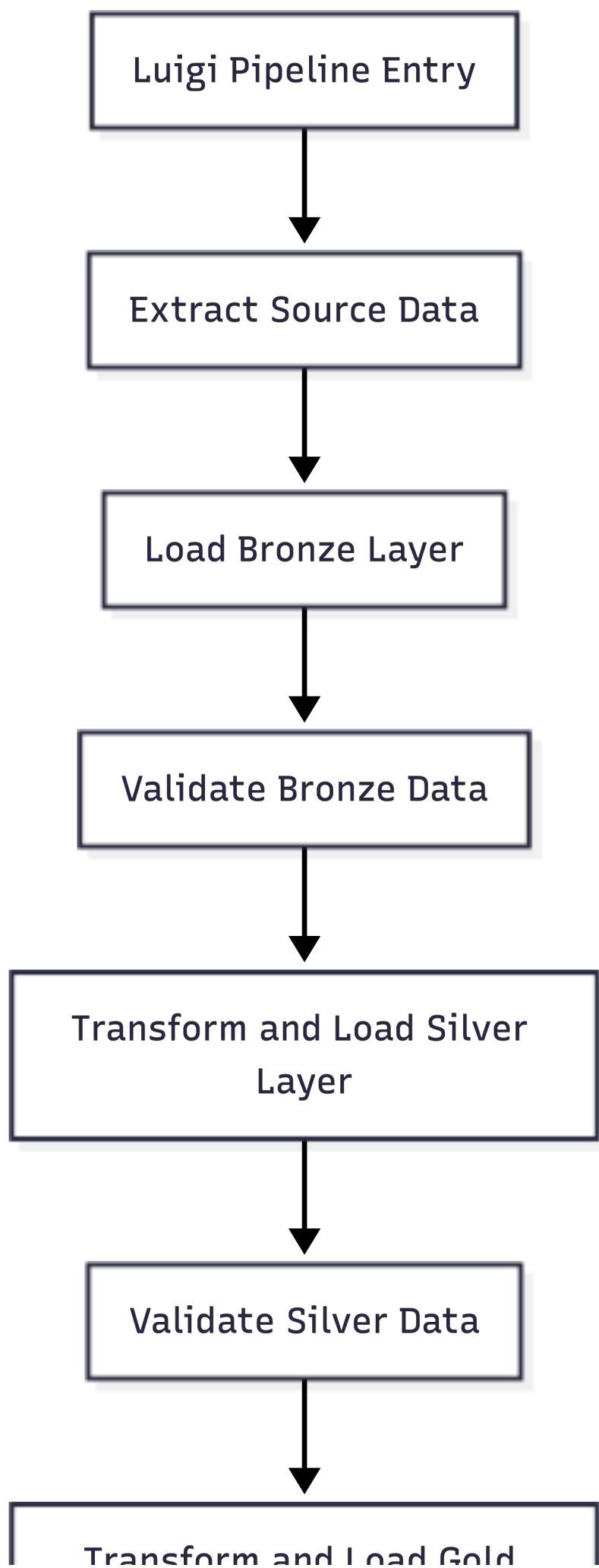
Bronze preserves source fidelity

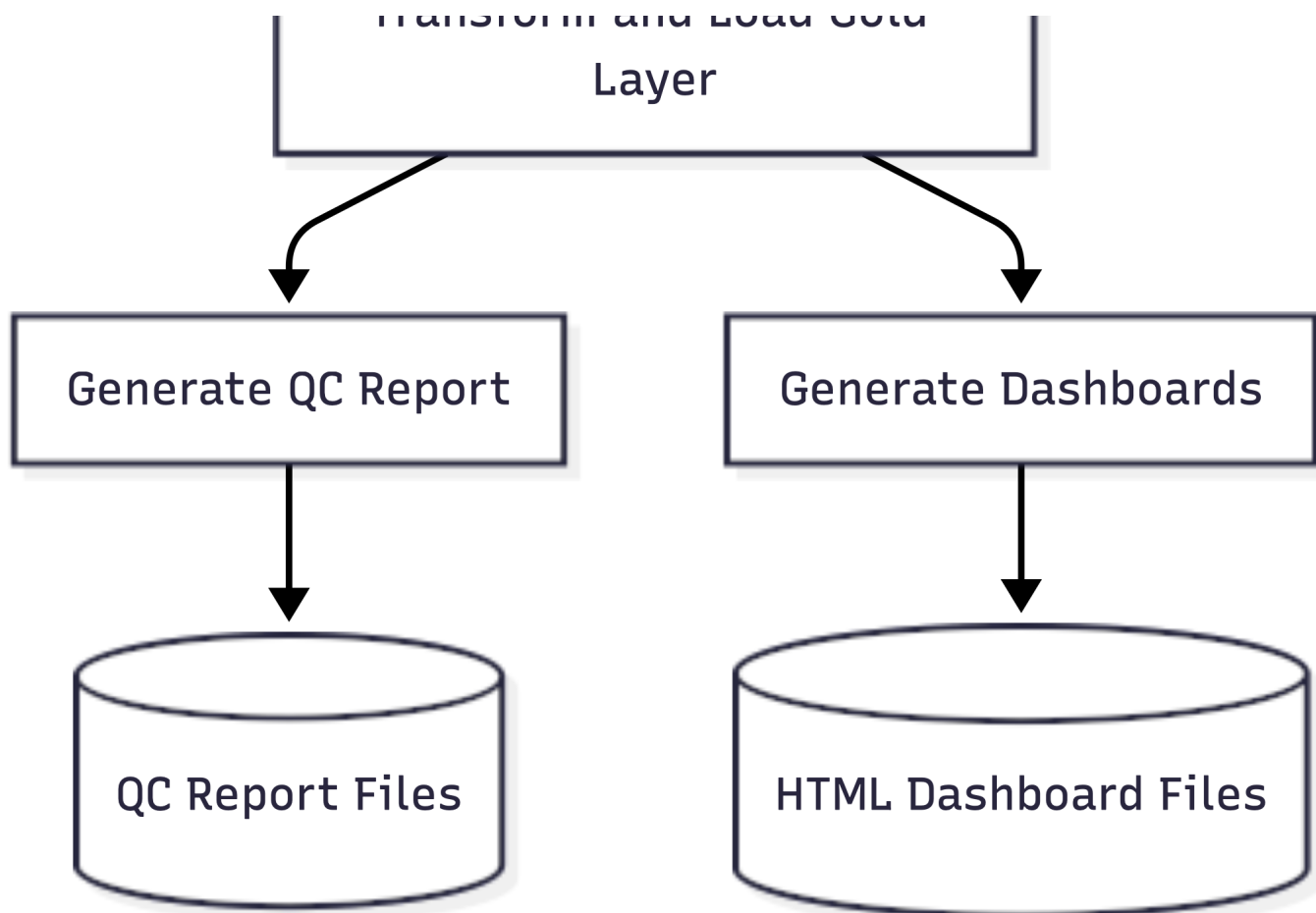
Bronze models intentionally keep many fields as strings. That prevents ingestion failures caused by malformed dates, inconsistent booleans, or partial records. Cleanup is deferred to silver, where the transformations:

- trim and normalize string values,
- parse dates and datetimes,
- infer booleans such as active status and per-diem flags,
- derive worked minutes, scheduled minutes, lateness, overtime, and work date,
- drop duplicate employee and timesheet records before loading constrained tables.

The trade-off is that bronze is a staging zone rather than an immediately query-friendly dataset.

Luigi markers and manifest hashing control reruns





The ETL uses Luigi `LocalTarget` outputs under `logs/markers/` and discovery manifests under `logs/manifests/`.

- Local discovery hashes file contents.
- MinIO discovery writes object names plus etags.
- Those hashes feed downstream task outputs.

This makes reruns deterministic: unchanged inputs do not force unnecessary work, while changed inputs produce new manifest signatures and new downstream markers. The trade-off is that orchestration state is filesystem-based rather than centrally managed.

Quality gates stop bad data before analytics

Bronze and silver validations run before gold metrics are built.

- Bronze checks ensure employee and timesheet rows have non-empty `client_employee_id` values.
- Silver checks enforce employee uniqueness, unique timesheet shifts, employee-to-timesheet referential integrity, and sane ranges for hours and dates.

That keeps broken source data from silently contaminating analytics. The trade-off is that the pipeline fails fast instead of partially producing outputs.

Gold metrics are materialized ahead of time

Gold tables such as `headcount_trend`, `employee_attendance_metrics`, and `department_monthly_metrics` are precomputed instead of being calculated live inside dashboards or API handlers. That makes reads simpler and faster. The trade-off is that metrics must be recomputed when source data changes.

Setup Instructions

Prerequisites

- Docker with Compose support (recommended)
- Or: Python 3.10+, PostgreSQL 15+, `pip` or `uv` for local setup

Docker setup (recommended)

The fastest way to run the full stack—ETL, database, MinIO, API, and visualizations—is via Docker:

```
docker compose up --build
```

This starts:

- `postgres`
- `minio`
- `api`

At container startup, the `api` service automatically:

1. Runs `alembic upgrade head` to initialize schemas
2. Uploads raw files from `data/raw/` to MinIO
3. Runs the full ETL pipeline (bronze → silver → gold)
4. Generates interactive dashboards via `dashboard/visualize.py`
5. Starts the API server

Service URLs:

Service	URL
API docs	<code>http://localhost:5173/docs</code>
MinIO console	<code>http://localhost:9001</code>

Shutdown:

```
docker compose down
```

To remove volumes as well:

```
docker compose down -v
```

Local setup

For development or when Docker is not available:

1. Create and activate a virtual environment

```
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
```

2. Install dependencies

```
uv sync
```

or

```
pip install -e .
```

3. Create the environment file

```
cp .env.example .env
```

Minimum local configuration:

```
DATABASE_URL=postgresql://hr_insights:hr_insights@localhost:5432/hr_insights
```

Common ETL settings:

```
HR_INSIGHTS_MINIO_ENDPOINT=localhost:9000
HR_INSIGHTS_MINIO_ACCESS_KEY=minioadmin
HR_INSIGHTS_MINIO_SECRET_KEY=minioadmin
HR_INSIGHTS_MINIO_SECURE=false
HR_INSIGHTS_MINIO_BUCKET=hr-insights
HR_INSIGHTS_RAW_DATA_DIR=data/raw
HR_INSIGHTS_MANIFESTS_DIR=logs/manifests
HR_INSIGHTS_ETL_BATCH_SIZE=1000
```

4. Start PostgreSQL

If you want to use Docker for just the database:

```
docker compose up -d postgres
```

5. Initialize schemas and tables

Either run migrations:

```
alembic upgrade head
```

or create schemas and tables directly:

```
python -m src.app.database
```

Both approaches create the **bronze**, **silver**, and **gold** schemas plus the mapped tables.

6. Add raw input files

Place raw CSV files in **data/raw/**.

Current ETL behavior expects:

- filenames starting with **employee** to load into **bronze.employee_raw**
- filenames starting with **timesheet** to load into **bronze.timesheet_raw**
- pipe-delimited files because the bronze loader reads with **sep="|"**

Files that do not match the expected prefixes are skipped by the bronze load task.

7. Optional: start MinIO and upload source files

If you want MinIO-backed ingestion:

```
docker compose up -d minio
```

Then upload local files into the configured bucket:

```
python src/etl/upload_to_minio.py data/raw
```

Usage Guide

ETL execution order

The full pipeline runs in this order:

1. discover files from local storage or MinIO,
2. extract them into `data/raw/` when needed,
3. load bronze raw tables,
4. validate bronze,
5. transform and load silver,
6. validate silver,
7. compute gold analytics,
8. write a QC report,
9. generate dashboard HTML files.

ETL commands

Run the full ETL, QC report, and dashboards from MinIO

```
python -m src.etl.run --module src.etl.quality.tasks RunDashboards --local-scheduler --all-months
```

Run only through the gold layer

```
python -m src.etl.run --module src.etl.gold.tasks LoadAllGold --local-scheduler --all-months
```

Run the QC report without dashboard generation

```
python -m src.etl.run --module src.etl.quality.tasks RunQCReport --local-scheduler --all-months
```

Outputs

After the pipeline runs, check:

Output	Location	Notes
Luigi manifests	<code>logs/manifests/</code>	discovered input files and hashes/etags
Luigi markers	<code>logs/markers/</code>	completion markers for bronze, silver, gold, QC, and dashboards
QC reports	<code>logs/reports/</code>	text reports such as <code>qc_report_YYYYMMDD_HHMMSS.txt</code>
Dashboards	<code>dashboard/output/</code>	Interactive Plotly HTML dashboards—see Visualizations

Generate dashboards directly

If gold tables already exist:

```
python dashboard/visualize.py
```

The script reads `DATABASE_URL` unless `--db` is provided explicitly.

Start the API locally

```
uvicorn src.app.main:app --reload --port 5173
```

Available docs:

- Swagger UI: <http://localhost:5173/docs>
- ReDoc: <http://localhost:5173/redoc>
- OpenAPI JSON: <http://localhost:5173/openapi.json>

Visualizations

The project produces **interactive Plotly HTML dashboards** that turn gold-layer analytics into clear, actionable insights. Each visualization now carries a short story callout and explicit drill-down controls so viewers can pivot quickly from the high level to the group they care about.

Highlights:

- Story panel on the workforce dashboard summarizes the latest headcount trend, turnover, and early attrition.
- Drill-down menu exposes per-organization, per-department, and per-job lenses without leaving the page.
- Filters and range sliders keep time navigation fast; tooltips are tuned for numeric readability.

Generated dashboards (current)

Dashboard	Purpose	Interactivity
workforce_trend.html	Headcount, turnover, early attrition over time	Story callout + org/dept/job drill-down + monthly/quarterly toggle + range slider
work_hours_overtime.html	Working-hours distribution and overtime pressure	Rolling-window toggle (7/14/30d) + hover tooltips
attendance_discipline.html	Late arrivals, early departures, top offenders	Comparative bars + linked heatmap output
attendance_heatmap.html	Calendar heatmap of late arrivals	Date/weekday heatmap for quick anomaly scanning

Note: Legacy entries ([executive_story.html](#), [department_story.html](#), etc.) are not yet implemented; build equivalents here if needed.

Output location

Dashboards are written to `dashboard/output/`. Open any `.html` file in a browser to explore interactively.

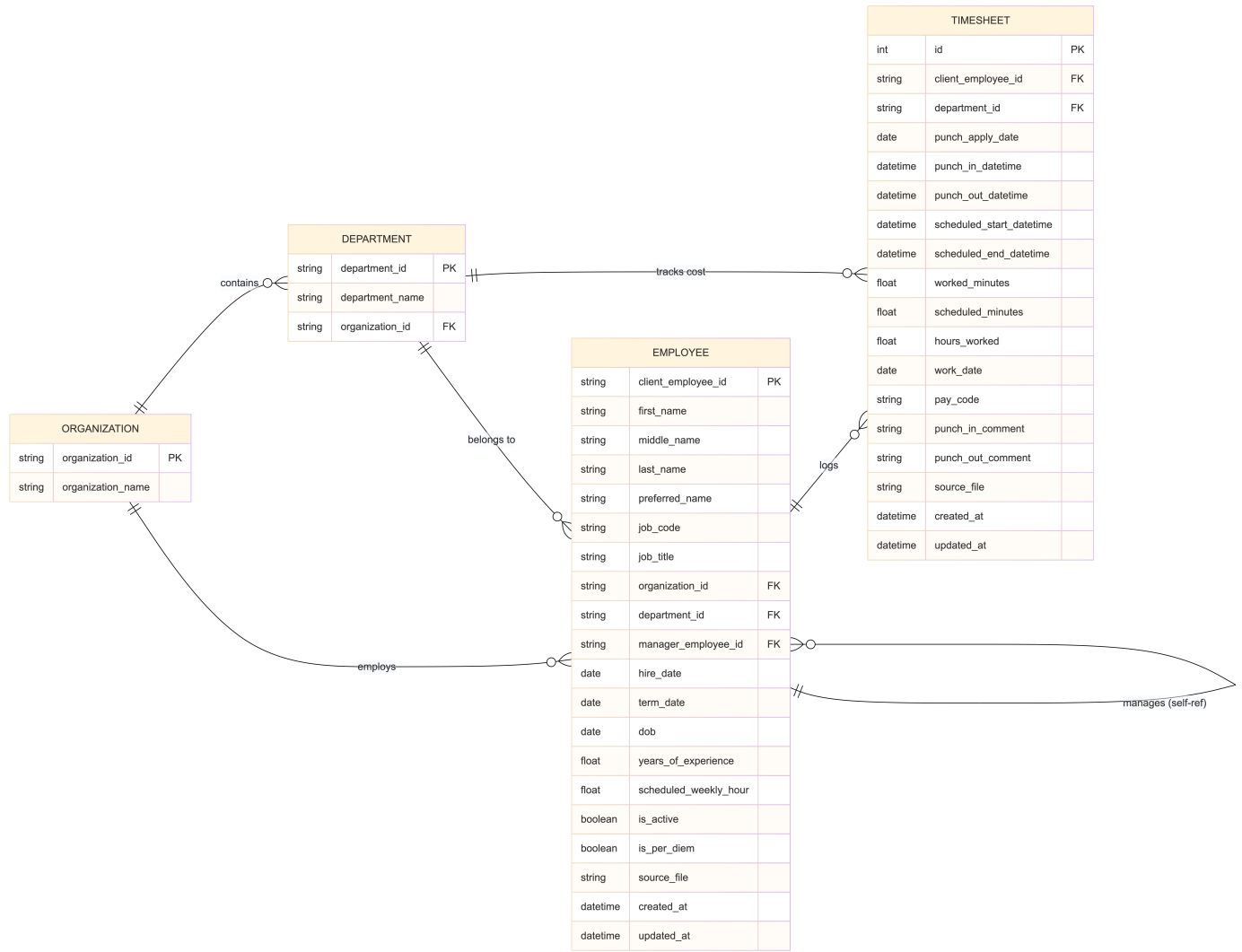
Regenerating dashboards

After the gold layer is populated:

```
python dashboard/visualize.py
```

Schema Documentation

Database model



Schema roles

Schema	Purpose	Design style
bronze	Raw landing zone for source files	Minimal transformation, preserves source fidelity
silver	Cleaned operational model	Typed, deduplicated, relational, API-friendly
gold	Reporting and KPI layer	Denormalized analytical marts

Core entity relationships

Bronze layer

Bronze is intentionally light on relationships.

- bronze.employee_raw: raw employee extracts plus source_file
- bronze.timesheet_raw: raw timesheet extracts plus source_file

These are staging tables rather than the authoritative relational model.

Silver layer

Silver contains the normalized operating model:

- silver.organization 1-to-many silver.department
- silver.organization 1-to-many silver.employee
- silver.department 1-to-many silver.employee
- silver.employee self-references silver.employee.client_employee_id through manager_employee_id
- silver.employee 1-to-many silver.timesheet
- silver.department is referenced by silver.timesheet.department_id

Key design details:

- `silver.employee.client_employee_id` is the primary employee business key.
- `silver.timesheet` enforces uniqueness on (`client_employee_id`, `punch_in_datetime`, `punch_out_datetime`).
- `silver.timesheet` is indexed by employee and work date for downstream analytics.
- timesheet rows for unknown employees are filtered out during loading and checked by validation.

Gold layer

Gold tables are derived from silver and optimized for analytical reads:

Table	Grain	Purpose
<code>gold.employee_monthly_snapshot</code>	one row per employee per month	active status, tenure, job, and department snapshot
<code>gold.timesheet_daily_summary</code>	one row per employee per day	worked hours, shifts, lateness, early departure, and overtime
<code>gold.employee_attendance_metrics</code>	one row per employee per month	attendance and working-hour KPIs
<code>gold.department_monthly_metrics</code>	one row per department per month	headcount, hires, terminations, turnover, and attendance rates
<code>gold.headcount_trend</code>	one row per month	organization-level headcount and attrition trend
<code>gold.organization_metrics</code>	one row per month	company-wide aggregates

Gold tables do not currently declare foreign keys back to silver. They are rebuilt analytical marts rather than normalized transactional tables.

Data lineage summary

1. Raw employee and timesheet CSV files land in local storage or MinIO.
2. Bronze stores them with minimal coercion and `source_file` lineage.
3. Silver normalizes organizations, departments, employees, and timesheets.
4. Gold computes daily, monthly, department, employee, and organization metrics.
5. Dashboards and analytics endpoints read from the resulting silver and gold tables.

Practical relationship notes

- `organization_id`, `department_id`, and `client_employee_id` are the main identifiers reused across layers.
- manager relationships only exist in silver because bronze does not assume referential correctness.
- gold attendance metrics depend on valid `work_date`, shift timings, and the lateness and overtime calculations performed during silver transformation.
- employee source changes can affect headcount and tenure outputs even if no timesheet data changes.

API Documentation

Interactive docs are available at `/docs` and `/redoc`. The raw OpenAPI schema is available at `/openapi.json`.

Authentication

All endpoints except `/health` and `POST /token` require a bearer token:

```
Authorization: Bearer <access_token>
```

Obtain a token:

- endpoint: `POST /token`
- body: `{"username": "<username>", "password": "<password>"}`
- response: `{"success": true, "data": {"access_token": "<jwt>", "token_type": "bearer"}, "message": "..."}>`

The repository includes in-memory demo users in `src/app/core/user_store.py` for local use. Replace or manage those credentials before using this outside a demo environment.

Authorization

- `admin`: full CRUD on employees plus read access to timesheets and analytics
- `viewer`: read-only access to employees, timesheets, and analytics

Response envelope

Every API response follows the same shape:

- `success`: boolean
- `message`: human-readable summary
- `data`: optional payload
- `error`: optional error detail

Endpoint summary

Area	Method	Path	Description
Auth	POST	<code>/token</code>	Exchange username and password for a JWT
Health	GET	<code>/health</code>	Service health check
Employees	POST	<code>/employees</code>	Create employee (<code>admin</code>)
Employees	GET	<code>/employees</code>	List employees
Employees	GET	<code>/employees/{employee_id}</code>	Get one employee
Employees	PATCH	<code>/employees/{employee_id}</code>	Update employee (<code>admin</code>)
Employees	DELETE	<code>/employees/{employee_id}</code>	Delete employee (<code>admin</code>)
Timesheets	GET	<code>/timesheets</code>	List timesheets
Timesheets	GET	<code>/timesheets/employee/{employee_id}</code>	List timesheets for one employee
Timesheets	GET	<code>/timesheets/filter</code>	Filter by date range and optional employee
Analytics	GET	<code>/analytics/headcount</code>	Headcount trend
Analytics	GET	<code>/analytics/departments</code>	Department metrics
Analytics	GET	<code>/analytics/employees/{employee_id}/attendance</code>	Employee attendance metrics
Analytics	GET	<code>/analytics/employees/attendance</code>	Attendance metrics for all employees
Analytics	GET	<code>/analytics/organization</code>	Organization-wide metrics
Analytics	GET	<code>/analytics/timesheets/daily</code>	Daily timesheet summary

Project Structure

Application code

- `src/app/main.py`: FastAPI app setup, router registration, and health endpoint
- `src/app/api/`: route handlers for auth, employees, timesheets, and analytics
- `src/app/core/`: config, logging, roles, JWT/password helpers, and user store
- `src/app/schemas/`: Pydantic request and response models
- `src/app/services/`: business logic separated from HTTP routes
- `src/app/database.py`: SQLAlchemy engine, session factory, and schema/table initialization

ETL code

- `src/etl/bronze/`: discovery, extraction, and raw table loading
- `src/etl/silver/`: cleaning logic and relational loads
- `src/etl/gold/`: KPI calculations and analytical loads
- `src/etl/quality/`: validation checks, QC reporting, and dashboard orchestration

- `src/etl/run.py`: Luigi entry point

Data models and assets

- `src/db/models/`: SQLAlchemy models for bronze, silver, and gold
- `dashboard/visualize.py`: dashboard generation entry point
- `dashboard/visualizations.py`: story-led KPI dashboards and drill-down views
- `images/`: architecture and schema diagrams
- `data/raw/`: local raw input files

Environment Variables

Variable	Default	Used by
DATABASE_URL	postgresql://hr_insights:hr_insights@localhost:5432/hr_insights	DB, ETL, dashboard, API
HR_INSIGHTS_MINIO_ENDPOINT	localhost:9000	ETL
HR_INSIGHTS_MINIO_ACCESS_KEY	minioadmin	ETL
HR_INSIGHTS_MINIO_SECRET_KEY	minioadmin	ETL
HR_INSIGHTS_MINIO_SECURE	false	ETL
HR_INSIGHTS_MINIO_BUCKET	hr-insights	ETL
HR_INSIGHTS_RAW_DATA_DIR	data/raw	ETL
HR_INSIGHTS_MANIFESTS_DIR	logs/manifests	ETL
HR_INSIGHTS_ETL_BATCH_SIZE	1000	ETL
HR_INSIGHTS_ETL_DEFAULT_YEAR	0	ETL
HR_INSIGHTS_ETL_DEFAULT_MONTH	0	ETL
API_HOST	0.0.0.0	API
API_PORT	5173	API
API_DEBUG	false	API
API_ENV_NAME	development	API
API_RELOAD	true	API
SECRET_KEY	your-secret-key-change-in-production	JWT signing
ALGORITHM	HS256	JWT signing
ACCESS_TOKEN_EXPIRE_MINUTES	30	JWT expiry

All of these can be set in a root `.env` file.

Analytics SQL

Prebuilt KPI queries for the gold layer live in `src/etl/gold/metrics/kpis.sql`.